Part 1 — Cloud Fundamentals
HPE SDE Interview Notes
This section covers the **cloud fundamentals** that form the base for the GCP questions that follow. The goal is not to memorize service definitions, but to understand the engineering trade-offs well enough to survive chained interview questions. This is especially important because the interview track is structured around the pattern **basic question → why → alternative → failure → scaling → security → troubleshooting**.

---

1. What is Cloud Computing?
Question: What is cloud computing?
Answer:
Cloud computing is the **on-demand delivery of computing resources**, such as compute, storage, networking, databases, and managed services, over a network without requiring the user to own and maintain the underlying physical infrastructure.
The important ideas are **on-demand provisioning, scalability, elasticity, and reduced infrastructure management**.
For example, instead of purchasing a physical server and manually installing an operating system, we can provision a virtual machine in Google Cloud when we need it and remove or resize it when the workload changes.

Follow-up: Why do companies use cloud computing?
The major engineering benefits are faster provisioning, elastic scaling, reduced infrastructure management, access to managed services, and the ability to deploy applications across different geographic locations.
However, **cloud does not automatically mean cheaper**. Poorly designed cloud architectures can become expensive.
For example, unnecessarily large VMs, excessive storage, uncontrolled network traffic, or unused resources can generate significant costs.
Therefore, cloud architecture involves balancing **cost, performance, reliability, security, and operational requirements**.

Follow-up: Is cloud always cheaper than on-premises infrastructure?
No.
Cloud can reduce capital expenditure and operational overhead, and it allows resources to scale according to demand. However, a poorly optimized cloud deployment can cost more than an appropriately designed on-premises system.
A strong interview answer is:
"Cloud provides flexibility and elastic resource provisioning, but cost efficiency depends on architecture and workload characteristics. Cloud cost optimization is itself an engineering concern."

---

2. IaaS, PaaS and SaaS
A common interview question is:
Question: What are IaaS, PaaS and SaaS?
The distinction is primarily about **how much of the infrastructure you manage versus how much the cloud provider manages**.
IaaS — Infrastructure as a Service
With IaaS, the provider gives you infrastructure such as virtual machines, CPU, memory, disks and networking.
You are still responsible for much of the software stack, including the operating system, runtime, application and configuration.
Google Compute Engine is an example of IaaS.
A simplified model is:
Application
Runtime
Operating System
———————
Cloud Provider
Hardware
Networking

Data Center
The important point is that **you get significant control, but also significant operational responsibility**.

PaaS — Platform as a Service
With PaaS, the provider manages substantially more of the underlying infrastructure and platform.
You primarily focus on your application.
Cloud Run is a fully managed platform for running containerized applications and is commonly described as PaaS-like in the traditional service-model classification.
Instead of managing a VM yourself, you provide your application/container and let the platform handle much of the infrastructure.

SaaS — Software as a Service
With SaaS, you consume a complete software product.
For example, **Google Workspace**.
You aren't responsible for managing the application's:

- Servers
- Operating system
- Runtime
- Infrastructure

You simply use the software.

Interview Shortcut
Remember:
IaaS → You manage more.
PaaS → Provider manages more.
SaaS → You mostly consume the finished product.

---

3. Cloud Run: PaaS or Serverless?
Question: Is Cloud Run PaaS or serverless?
Don't get trapped trying to force it into one category.
A good answer is:
"Cloud Run is a fully managed serverless platform for running containers. In the traditional IaaS/PaaS/SaaS classification, it is generally considered PaaS-like because Google manages the underlying infrastructure."
The important concept is that **serverless doesn't mean there are no servers**.
Servers still exist. You simply don't manage them directly.

---

4. Scalability
Question: What is scalability?
Scalability is the ability of a system to handle increasing workload by adding resources.
There are two fundamental approaches.
Vertical Scaling
Increase the capacity of an existing machine.
For example:
4 CPU / 8 GB RAM
↓
16 CPU / 64 GB RAM
This is called **scaling up**.
The advantage is simplicity because you're still dealing with one machine.
The disadvantage is that the machine has a physical or platform limit, and it remains a single failure point unless additional redundancy exists.

Horizontal Scaling
Add more instances.

Server 1
Server 2
Server 3
Server 4
This is called **scaling out**.
Horizontal scaling is especially important for distributed systems because workload can be distributed across multiple instances.

Interview Question: Which is better?
There is no universal answer.
Vertical scaling is often simpler and may be appropriate for workloads that are difficult to distribute.
Horizontal scaling generally provides better scalability and availability, but requires the application to support distributed execution.
For example, a **stateless web API** is usually a good candidate for horizontal scaling.

---

5. Scalability vs Elasticity
These two are easy to confuse.
Question: What is elasticity?
Elasticity is the ability of a system to dynamically increase or decrease resources as workload changes.
For example:
Normal traffic
↓
3 instances

Traffic increases
↓
10 instances

Traffic decreases
↓
3 instances
The system automatically adapts to demand.
Difference
Scalability asks:
Can the system handle more load?
Elasticity asks:
Can the system dynamically adjust capacity as the load changes?
Cloud platforms are particularly useful for elasticity because resources can be provisioned and released dynamically.

---

6. High Availability
Question: What is high availability?
High availability means designing a system so that it can **continue providing service despite failures of individual components**.
Consider:
Client
|
v
Server
If the server fails, the application is unavailable.
A highly available architecture might instead use:

```
        +-- Server 1
Client --> LB +-- Server 2
        +-- Server 3
```

If Server 2 fails, the other instances can continue serving requests.
The important idea is **redundancy**.

Follow-up: Does having multiple servers automatically provide high availability?
No.
If all servers are located in the same failure domain, one infrastructure failure could still take everything down.
For example:

```
Region
  |
  +-- Zone A
     +-- Server 1
     +-- Server 2
     +-- Server 3
```

If Zone A fails, all three servers may become unavailable.
Therefore, production systems often distribute workloads across multiple failure domains.

---

7. Availability vs Reliability
Question: What's the difference between availability and reliability?
Availability asks:
"Is the system accessible when users need it?"
Reliability asks:
"Does the system consistently behave correctly over time?"
For example, an API might return HTTP 200 for every request but return incorrect data half the time.
It is technically **available**, but it is not particularly **reliable**.
This distinction becomes important in distributed systems and production monitoring.

---

8. Region vs Zone
This is important for GCP interviews.
Question: What is a region?
A **region** is a geographic area containing multiple zones.
Question: What is a zone?
A **zone** is an isolated deployment area within a region.
Conceptually:

```
Google Cloud
   |
   +-- Region
       |
       +-- Zone A
       +-- Zone B
       +-- Zone C
```

The purpose of having multiple zones is to provide isolation from failures affecting a particular zone.

Follow-up: Why deploy an application across multiple zones?
Suppose your application is running only in Zone A.
If Zone A experiences a major infrastructure failure, your application could become unavailable.
If you have:
Zone A → Application Instance
Zone B → Application Instance
Zone C → Application Instance
then a failure in one zone doesn't necessarily bring down the entire application.
This improves **fault tolerance and availability**.

Follow-up: Why deploy across multiple regions?

Multiple regions provide protection against larger geographic or regional failures and can also help reduce latency for geographically distributed users.
However, multi-region architecture introduces additional complexity:

- Data replication
- Consistency
- Network latency
- Failover
- Cost

Therefore:
More regions ≠ automatically better architecture.
You use them when the availability, disaster-recovery, latency, or geographic requirements justify the additional complexity.

---

9. Managed Services
Question: What is a managed service?
A managed service is a service where the cloud provider takes responsibility for much of the underlying infrastructure and operational work.
Consider a database.
With a self-managed database on a VM, you may need to handle:
VM
OS
Database installation
Patching
Backups
Monitoring
Recovery
With a managed database such as **Cloud SQL**, Google handles much of the underlying infrastructure management, allowing you to focus more on the database itself, its schema, queries, configuration, and application integration.

Follow-up: Does managed mean I don't need to worry about the database?
No.
Managed services reduce infrastructure responsibilities; they don't eliminate engineering responsibilities.
You still need to think about:

- Database schema
- Query performance
- Indexes
- Connection management
- Credentials
- IAM
- Capacity
- Backups and recovery requirements
- Application behavior
- Cost

A good interview statement is:
"Managed means reduced infrastructure responsibility, not zero operational responsibility."

---

10. Choosing a GCP Compute Service
This is where the concepts start connecting.
Question: "I have a Python web API. Where would you deploy it?"
Don't immediately name a service.
First determine the requirements:
Is the application stateless?
Does it require OS-level control?

Does it need Kubernetes?
How much infrastructure do we want to manage?
Does it need specialized hardware?
What are the scaling requirements?
For a straightforward stateless HTTP API packaged as a container, **Cloud Run** is often an attractive choice because Google manages the infrastructure and scaling.

Follow-up: Why not Compute Engine?
A strong answer:
"Compute Engine would give me more control, but I would also have to manage the VM, operating system, runtime and more of the infrastructure. For a stateless HTTP service where I don't need VM-level control, Cloud Run gives me a higher-level abstraction and reduces operational overhead."

Follow-up: Why not GKE?
A strong answer:
"GKE makes more sense when I need Kubernetes-specific capabilities or more control over container orchestration, networking and workload management. For a simple stateless service, GKE could introduce unnecessary operational complexity."
GKE and Cloud Run solve overlapping but different problems: GKE provides Kubernetes-based orchestration, while Cloud Run provides a more managed environment for running containers.

---

11. What Happens When Traffic Increases?
Question:
"Your API suddenly receives 100 times its normal traffic. What do you do?"
Don't simply answer:
"Add more servers."
Think through the system.
For a horizontally scalable application:
Traffic increases
↓
Application instances increase
↓
Requests distributed
↓
Database/cache/external dependencies
↓
Monitor bottlenecks
The important follow-up is:
"What if the application scales successfully but the database doesn't?"
That is where many beginner answers fall apart.
Suppose one backend instance maintains 100 database connections.
You scale:
1 backend → 10 backends
You could potentially create:
10 × 100 = 1000 connections
If the database can only handle 300 connections, scaling the application actually makes the system **less stable**.
Therefore, distributed-system scaling must consider the **entire dependency chain**, not just the frontend application.

---

12. The Most Important Mental Model
For an SDE interview, don't think:
"Which GCP service should I use?"
Think:
"What requirement am I trying to satisfy?"

For example:
Need VM-level control
↓
Compute Engine

Need managed container execution
↓
Cloud Run

Need Kubernetes orchestration
↓
GKE

Need relational database
↓
Cloud SQL

Need object storage
↓
Cloud Storage
Then ask:
What are the trade-offs?
That is the level at which you should discuss cloud architecture in an SDE interview.

Rapid Revision — Part 1
Q: What is cloud computing?
On-demand access to computing resources and managed services without owning the underlying physical infrastructure.
Q: IaaS?
Infrastructure such as VMs, networking and storage; you manage more of the software stack.
Q: PaaS?
A managed application platform where the provider handles substantially more infrastructure.
Q: SaaS?
A complete software product consumed by the user.
Q: Compute Engine?
Google Cloud's VM/IaaS service.
Q: Cloud Run?
A fully managed serverless platform for running containers.
Q: Scalability?
Ability to handle increasing workload by adding resources.
Q: Vertical scaling?
Increasing the capacity of an existing machine.
Q: Horizontal scaling?
Adding more machines/instances.
Q: Elasticity?
Dynamically adding and removing resources according to workload.
Q: High availability?
Designing the system to continue operating despite failures of individual components.
Q: Region?
Geographic area containing multiple zones.
Q: Zone?
An isolated deployment area within a region.
Q: Managed service?

A service where the provider handles much of the underlying infrastructure and operational work.

Q: Is cloud automatically cheaper?

No. Cost depends on architecture and workload.

Q: Is managed service completely hands-off?

No. It reduces infrastructure management but doesn't eliminate application/database/security responsibilities.

---

1. What is Google Compute Engine?

Question: What is Compute Engine?

Answer:

Google Compute Engine (GCE) is Google Cloud's **Infrastructure-as-a-Service (IaaS)** offering for running virtual machines on Google's infrastructure.

You choose resources such as:

- CPU
- Memory
- Storage
- Operating system
- Networking configuration

and Google provides the underlying physical infrastructure.

The key distinction is that **you have substantial control over the VM**, unlike a more managed platform such as Cloud Run. Google describes Compute Engine as providing self-managed VMs and bare-metal instances.

---

2. What is a Virtual Machine?

Question: What exactly is a VM?

A virtual machine is an isolated software environment that behaves like a physical computer.

A physical server can run multiple VMs:

```
Physical Server
    |
    +-- VM 1
    |
    +-- VM 2
    |
    +-- VM 3
```

Each VM can have its own:

- Operating system
- CPU allocation
- Memory
- Disk
- Network interfaces
- Processes

The virtualization layer responsible for creating and managing these VMs is called a **hypervisor**.

---

3. What is a Hypervisor?

Question: What does the hypervisor do?

The hypervisor is the layer that allows multiple virtual machines to share the underlying physical hardware while remaining logically isolated.

Conceptually:

```
VM 1      VM 2      VM 3
 |         |         |
 +----------+----------+
          |
      Hypervisor
```

|

Physical Hardware

The hypervisor allocates physical resources to virtual machines and maintains isolation between them.

For an SDE interview, you generally don't need to discuss the implementation details of Google's virtualization stack unless specifically asked.

---

4. Why Use a VM Instead of a Physical Server?

Question: Why would a company use Compute Engine instead of buying physical servers?

The major advantages are:

Provisioning speed: A VM can be created much faster than purchasing and configuring physical hardware.

Elasticity: Resources can be increased or additional instances can be created as demand changes.

Isolation: Different workloads can run in separate VM environments.

Operational flexibility: Machine types, disks and networking can be configured according to workload requirements.

Global deployment: Instances can be deployed in different Google Cloud regions and zones.

The important interview concept is:

Cloud doesn't eliminate infrastructure; it abstracts the physical infrastructure and gives you programmable access to it.

---

5. Machine Types

Question: What determines how powerful a Compute Engine VM is?

Primarily its **machine type**, which determines resources such as CPU and memory.

Conceptually:

Small VM
2 vCPU
4 GB RAM

vs.

Large VM
16 vCPU
64 GB RAM

Different workloads need different resource profiles.

For example, a lightweight API server may require relatively little CPU and memory, while a computational workload may need significantly more CPU.

Google provides different machine families optimized for different workload characteristics.

Follow-up: Should I always choose the largest machine?

No.

That would be poor cloud engineering.

A larger machine may provide more capacity but also costs more.

The goal is to choose an appropriate machine type based on:

- CPU utilization
- Memory requirements
- Network requirements
- Application behavior
- Expected workload
- Cost

This is another example of the cloud trade-off between **performance and cost**.

---

6. VM Boot Disk vs Persistent Storage

A VM needs storage.

There are two concepts you should distinguish.

Boot Disk

The boot disk contains the operating system and is used to start the VM.
For example:
Boot Disk
  |
  +-- Linux OS
  +-- System files
  +-- Installed software

Additional Persistent Disk

You can attach additional storage for application data.
VM
 |
 +-- Boot Disk
 |
 +-- Data Disk

This separation is useful because the operating system and application data often have different lifecycle requirements.

---

7. What Does "Persistent" Mean?

Question: Why is persistent storage important?

Persistent storage is designed so that data survives the lifecycle of the VM itself.
For example, suppose:
VM
 |
 +-- Application data

You delete the VM but retain the persistent disk.
You can attach that disk to another VM and recover the data.
This is fundamentally different from storing important data only in temporary runtime state.

---

8. VM Lifecycle

A VM isn't simply "running" or "dead."
Conceptually, an instance can be:
Provisioned
  ↓
Running
  ↓
Stopped
  ↓
Started again
  ↓
Deleted

Stopped vs Deleted

This distinction matters.
Stopped VM: The VM is no longer actively running, but its persistent resources can remain.
Deleted VM: The instance itself is removed. Whether associated storage is also removed depends on how the resources were configured.
An interviewer may ask:
"If you stop a VM, does its disk disappear?"
Not necessarily. Persistent disks are separate resources and can survive independently depending on the deletion configuration.

---

9. Public IP vs Private IP

This is extremely important because it connects Compute Engine to VPC networking.

A VM can have a **private IP address** for communication within its network and potentially a **public/external IP address** for communication through the internet.

Conceptually:

Internet
  |
Public IP
  |
 VM
  |
Private IP
  |
Internal Network

Private IP

Used for internal communication.

For example:

Backend VM
  |
  | private network
  ↓
Database

Public IP

Used when the VM needs external connectivity through an internet-facing address.

---

10. Why Should You Avoid Giving Every VM a Public IP?

Question:

"If private networking exists, why expose VMs publicly?"

Because every publicly reachable endpoint increases the attack surface.

Suppose you have:

Internet
  |
  +-- Web Server
  |
  +-- Backend
  |
  +-- Database

A better architecture is often:

Internet
  |
Load Balancer
  |
Web / API
  |
Private Network
  |
Database

The database doesn't need to be directly reachable from the internet.

This is where **network segmentation and least privilege** become important.

---

11. Firewall Rules

Question: How do you control traffic to a Compute Engine VM?

Through **VPC firewall rules**.

A firewall rule determines whether specific traffic should be allowed or denied based on characteristics such as:

- Direction

- Protocol
- Port
- Source
- Destination
- Applicable instances/tags or identities depending on configuration

For example:
Allow TCP
Port: 443
Source: Internet
allows HTTPS traffic.

Interview Scenario
"I deployed an Nginx server on a Compute Engine VM. The process is running, but I can't access it from my browser. What do you check?"
Don't immediately blame Nginx.
Troubleshoot layer by layer:
Browser
↓
DNS
↓
Public IP
↓
VPC Firewall
↓
VM Network Interface
↓
Port 443/80
↓
Nginx
You would check:

1. Is the VM running?
2. Does it have the expected IP?
3. Is the application actually listening on the expected port?
4. Is the firewall allowing that port?
5. Is the service bound to the correct interface?
6. Is there any load balancer or routing issue in front of it?

That is a much stronger answer than:
"Restart the VM."

---

12. SSH Access
Question: How do you connect to a Linux Compute Engine VM?
Typically through **SSH**.
Conceptually:
Your Computer
|
| SSH
↓
Compute Engine VM
You authenticate using credentials/keys and then obtain a shell on the VM.
Once connected, you're essentially operating a Linux server:
ps
top
systemctl

ss
ip addr
journalctl
This is why your Linux knowledge matters directly to cloud interviews.

---

13. Why SSH Is a Security Concern
If you expose SSH directly to the entire internet:
0.0.0.0/0 → TCP 22
you've significantly increased your attack surface.
A better design restricts administrative access through controlled mechanisms, such as limited source ranges, identity-aware access mechanisms, or a controlled administrative path.
The general security principle is:
Don't expose a service to more networks or users than necessary.
This is simply **least privilege applied to networking**.

---

14. Managed Instance Groups
Now we move from **one VM** to **multiple VMs**.
Suppose you have:
VM 1
VM 2
VM 3
VM 4
Managing each VM individually becomes difficult.
A **Managed Instance Group (MIG)** lets you manage a group of VM instances based on a common configuration/template.
Instead of manually creating every instance:
Create VM 1
Create VM 2
Create VM 3
...
you define the desired configuration and let the platform manage the group.

---

15. Why Do We Need Instance Groups?
Consider a web application.
Client
  |
Load Balancer
  |
  +-- VM 1
  +-- VM 2
  +-- VM 3
If VM 2 crashes, you don't want an engineer manually logging in at 3 AM to create another VM.
The managed instance group can help maintain the desired number of instances.
This leads directly to **autohealing**.

---

16. Autohealing
Question: What happens if one VM becomes unhealthy?
A managed instance group can use a **health check** to determine whether an instance is functioning correctly.
Conceptually:
Health Check
   |
   +-- VM 1 → Healthy
   +-- VM 2 → Unhealthy

+-- VM 3 → Healthy

If an instance remains unhealthy, the system can recreate/replace it according to the configured autohealing behavior.

This is an important distinction:

Autohealing is about recovering from unhealthy instances.

It is different from autoscaling.

---

17. Autoscaling vs Autohealing

This is a very likely interview question.

Autoscaling

Changes the **number of instances** according to workload.

Example:

Low traffic

↓

3 instances

High traffic

↓

10 instances

Autohealing

Replaces instances that are unhealthy.

Example:

3 instances

↓

1 becomes unhealthy

↓

Replace unhealthy instance

↓

3 healthy instances

Remember:

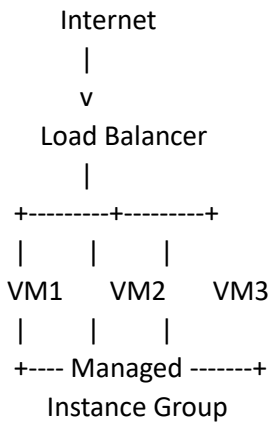
Autoscaling responds to demand. Autohealing responds to failure.

---

18. Load Balancer + Managed Instance Group

Now combine everything.

A typical architecture could look like:



The load balancer distributes traffic across healthy instances.

The managed instance group maintains the desired fleet.

Health checks determine whether instances are healthy.

Autoscaling changes the number of instances based on workload.

Autohealing replaces unhealthy instances.

This is the foundation of a highly available VM-based web application.

---

19. Complete Failure Scenario

Interviewer:

"You have three Compute Engine instances behind a load balancer. One instance crashes. What happens?"

A strong answer is:

"The health check should detect that the instance is unhealthy. The load balancer should stop sending traffic to that unhealthy instance, while the remaining healthy instances continue serving requests. If the instances are managed by a managed instance group with autohealing enabled, the unhealthy instance can be recreated to restore the desired capacity."

That's a **very strong interview answer** because it connects four concepts:

Health check → Load balancer → Autohealing → Availability

---

20. Another Failure Scenario

Interviewer:

"Traffic increases dramatically. What happens?"

A good answer:

"If the application is configured for horizontal autoscaling, the managed instance group can increase the number of VM instances based on the configured scaling policy. The load balancer then distributes incoming traffic across the available healthy instances."

Then expect the next attack:

"What if the database becomes the bottleneck?"

Your answer:

"Adding application instances doesn't automatically solve a database bottleneck. I'd inspect database CPU, memory, connection count, query latency and throughput. Depending on the workload, I might optimize queries, introduce caching, increase database capacity, use read replicas, or change the architecture."

That is the kind of answer that demonstrates **SDE-level reasoning rather than GCP memorization**.

---

21. Compute Engine vs Cloud Run

Question: When would you choose Compute Engine over Cloud Run?

Choose Compute Engine when you need more control over the underlying environment.

Examples include applications that require:

- OS-level customization
- Specialized system configuration
- Long-running VM workloads
- Specific networking behavior
- Software that doesn't fit comfortably into a managed container platform
- More direct control over the machine

Choose Cloud Run when you have a containerized application and want a highly managed execution environment with minimal infrastructure management.

The fundamental trade-off is:

Compute Engine gives you more control; Cloud Run gives you more abstraction and less operational responsibility.

---

22. Compute Engine vs Kubernetes/GKE

Question: Why not simply run everything on Compute Engine?

You can, but once you have many containerized services and complex orchestration requirements, manually managing containers across VMs becomes difficult.

GKE provides Kubernetes orchestration capabilities such as:

- Pod scheduling
- Deployments
- Service discovery
- Rolling updates
- Kubernetes-based workload management

So the decision is approximately:

Need VM

↓
Compute Engine

Need managed container execution
↓
Cloud Run

Need Kubernetes orchestration
↓
GKE
The exact choice depends on workload and operational requirements rather than the assumption that one service is universally better.

---

23. Compute Engine Interview Rapid Revision
What is Compute Engine?
Google Cloud's IaaS service for running VMs and, in some cases, bare-metal workloads.
What is a VM?
An isolated virtual computer running on shared physical infrastructure through virtualization.
What does a hypervisor do?
It manages virtual machines and allocates physical resources to them while maintaining isolation.
What is a machine type?
A configuration defining the VM's compute resources, such as CPU and memory.
What is a boot disk?
The disk containing the operating system used to boot the VM.
What is a persistent disk?
Persistent block storage that can outlive the VM instance depending on its configuration.
Public IP vs private IP?
A public IP provides external network reachability; a private IP is primarily used for internal network communication.
How do you control VM traffic?
Through VPC firewall rules.
What is a Managed Instance Group?
A managed collection of VM instances maintained according to a common configuration and desired state.
Autoscaling vs autohealing?
Autoscaling changes the number of instances based on demand.
Autohealing replaces unhealthy instances.
What does a health check do?
Determines whether an instance/service is healthy enough for the configured system to treat it as operational.

Part 3 — Google Cloud VPC & Networking
HPE SDE Interview Notes
This is one of the **highest-value sections** for your HPE interview because it connects directly to the Computer Networks preparation you've already done. The important thing is not memorizing GCP networking terminology; it is understanding how a VM gets an IP, how traffic reaches it, how traffic leaves it, and how access is controlled.

---

1. What is a VPC?
Question: What is a VPC?
Answer:
A **Virtual Private Cloud (VPC)** is a logically isolated virtual network in the cloud where resources such as Compute Engine VMs can communicate with each other and with external networks according to configured networking and security rules.
In Google Cloud, a VPC provides networking constructs such as:

- IP address ranges
- Subnets

- Routes
- Firewall rules
- Network connectivity

Think of it as the **network environment in which your cloud resources communicate**.

---

2. Why Do We Need a VPC?

Suppose you have:

Frontend VM

Backend VM

Database

You need to determine:

- Which IP addresses they use
- How they communicate
- Which traffic is allowed
- How traffic reaches the internet
- How external users reach the application

A VPC provides the networking foundation for all of this.

---

3. VPC vs Subnet

This is a very common interview question.

Question: What is the difference between a VPC and a subnet?

A **VPC is the overall virtual network**, while a **subnet is an IP address range within that network**.

Conceptually:

```
VPC
 |
 +-- Subnet A
 |    10.0.1.0/24
 |
 +-- Subnet B
     10.0.2.0/24
```

The subnet determines the IP range from which resources in that subnet receive addresses.

---

4. What is an IP Address Range?

Suppose you create:

10.0.1.0/24

This gives you an IPv4 address range.

You already know the subnetting concept from networking:

/24

=

255.255.255.0

So the subnet provides a defined pool of addresses that resources can use.

For example:

10.0.1.1

10.0.1.2

10.0.1.3

...

The exact usable-address behavior depends on the cloud networking implementation and reserved addresses, so don't blindly assume every address in the CIDR can be assigned to a VM.

---

5. What is a Route?

Question: What is a route?

A route tells the network **where traffic destined for a particular IP range should be sent**.

Conceptually:

Destination
|
v
Routing Table
|
v
Next Hop
For example, traffic destined for:
10.0.2.0/24
might be routed toward the network containing that subnet.
This is the same fundamental routing concept you learned in Computer Networks.

---

6. Default Route to the Internet
A common concept is a route for traffic destined for external networks.
Conceptually:
0.0.0.0/0
means:
Any IPv4 destination that doesn't match a more specific route.
This is the **default route**.
So if a VM wants to communicate with an external IP and there isn't a more specific route, the default route can determine how that traffic leaves the VPC.

---

7. Public IP vs Private IP
This is one of the most important cloud networking questions.
Question: What's the difference between a public IP and a private IP?
A **private IP** is used for communication within private/internal networks.
A **public/external IP** provides internet-facing connectivity.
For example:
Internet
|
Public IP
|
Web Server
|
Private IP
|
Backend
|
Private IP
|
Database
You generally don't want your database directly exposed to the internet.

---

8. Why Use Private IPs?
Suppose you have:
Internet
|
+-- Web Server
|
+-- Database
This unnecessarily exposes the database.
A better architecture is:
Internet

|
Load Balancer
|
Web/API
|
Private Network
|
Database
The database can communicate internally without being directly reachable from the public internet.
This reduces the attack surface.

---

9. VPC Firewall Rules
Question: How do you control which network traffic can reach a VM?
Through **VPC firewall rules**.
A firewall rule can control traffic based on characteristics such as:

- Source
- Destination
- Protocol
- Port
- Direction
- Which resources the rule applies to

For example:
Allow
TCP
Port 443
allows HTTPS traffic when the rule's other conditions match.

---

10. Ingress vs Egress
This is another easy interview question.
Ingress
Traffic **coming into** a resource.
Internet
|
v
VM
The traffic entering the VM is ingress.
Egress
Traffic **leaving** a resource.
VM
|
v
Internet
The traffic leaving the VM is egress.
Remember:
Ingress = in. Egress = out.

---

11. Firewall Scenario
Interviewer:
"Your application is listening on port 8080, but users cannot connect. What do you check?"
Don't immediately assume the application is broken.
Check the path:
Client
|

v
Public IP / Load Balancer
|
v
Firewall
|
v
VM
|
v
Port 8080
|
v
Application
You need to determine where the packet is being stopped.
Check:

1. Is the VM running?
2. Does it have the expected IP?
3. Is the application listening on port 8080?
4. Is it listening on the correct interface?
5. Does the VPC firewall allow the traffic?
6. Is there a routing problem?
7. If there is a load balancer, is the backend considered healthy?

This is exactly how you should approach cloud networking troubleshooting: **follow the packet path**.

---

12. Network Security Principle
Question: Should you allow 0.0.0.0/0 to everything?
Absolutely not.
0.0.0.0/0 represents all IPv4 addresses.
Allowing:
0.0.0.0/0 → TCP 22
means SSH is reachable from anywhere, subject to the rest of the configuration.
That's a much larger attack surface than necessary.
A better principle is:
Allow only the traffic that is actually required, from the smallest appropriate source range.
This is the networking form of **least privilege**.

---

13. VPC and Internet Access
A VM with a private IP doesn't automatically mean:
"It can freely communicate with the public internet."
You need appropriate networking configuration.
For outbound internet access, a private VM commonly uses **Cloud NAT** rather than receiving a public IP.
Conceptually:
Private VM
|
v
Cloud NAT
|
v
Internet
The VM can initiate outbound connections without needing its own public IP.

---

14. Why is Cloud NAT Useful?

Suppose your backend VM needs to:
apt update
download packages
call an external API
access a public repository
But you don't want the VM directly exposed through a public IP.
You can use:
Private VM
  |
  v
Cloud NAT
  |
  v
Internet
This gives the VM outbound internet connectivity while avoiding the need for a public IP on the VM.

---

15. Important Distinction: Inbound vs Outbound
This is a common follow-up.
Private VM + NAT
The VM can initiate:
VM → Internet
But NAT does **not** mean:
Internet → VM
is automatically allowed.
That's an important security distinction.

---

16. Internal Communication
Suppose you have:
Frontend
Backend
Database
They can communicate using internal/private addressing.
For example:
Frontend
  |
  | private network
  v
Backend
  |
  | private network
  v
Database
This is preferable to routing internal application traffic through the public internet.

---

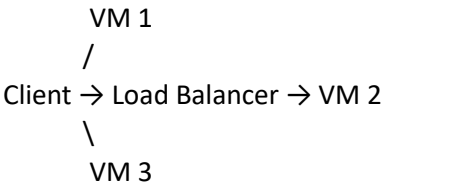
17. DNS in Cloud Environments
Question: How does a backend find a database?
You don't want to hardcode an IP address everywhere.
Instead, applications commonly use **DNS names**.
Conceptually:
backend.internal
    |
    v
DNS



Private IP
This gives you an abstraction between:
Application
and:
Actual IP address
This becomes particularly important when instances are dynamically created or replaced.

---

18. Load Balancer
Now connect networking with Compute Engine.
Suppose you have:
VM 1
VM 2
VM 3
Instead of giving users three different addresses, expose a load balancer:



The load balancer distributes incoming traffic across backend instances according to its configuration.

---

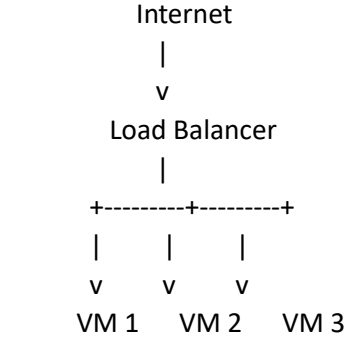
19. Why Do We Need a Load Balancer?
There are three major reasons.
Scalability: Traffic can be distributed across multiple instances.
Availability: If one backend becomes unhealthy, traffic can be directed toward healthy backends.
Single entry point: Clients don't need to know the individual backend instance addresses.

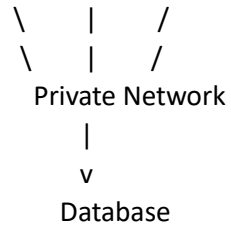
---

20. Health Checks and Load Balancing
This is an important chained question.
Interviewer:
"How does the load balancer know which VM is healthy?"
Through **health checks**.
For example, the load balancer might periodically test an endpoint such as:
GET /health
If the backend responds appropriately:
Healthy
If it repeatedly fails:
Unhealthy
Traffic can then be directed toward healthy instances.

---

21. Complete GCP Web Architecture
At this point, we can combine almost everything we've learned:





Firewall rules control which traffic is allowed.
Health checks determine backend health.
Managed Instance Groups maintain the VM fleet.
Autoscaling adjusts the number of instances.
Private networking protects internal services from direct internet exposure.
This is the basic architecture you should be able to explain verbally.

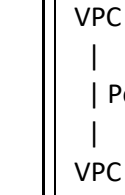
---

22. VPC Peering

Question: What if two separate VPC networks need to communicate?

One option is **VPC Network Peering**.

Conceptually:



This allows resources in the two networks to communicate using private IP connectivity, subject to routing and firewall configuration.

The important point is:

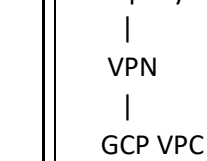
Peering connects networks privately; it does not make them one single VPC.

---

23. VPN

Another way to connect networks is a VPN.

For example:



This is useful when an organization wants private connectivity between on-premises infrastructure and cloud resources.

For an SDE interview, know the concept; you don't need to memorize every GCP VPN configuration option.

---

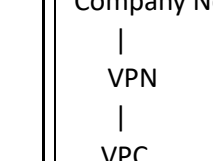
24. VPC vs VPN

Very common confusion.

VPC is the virtual network itself.

VPN is a mechanism for creating a secure network connection between networks.

For example:



---

25. Troubleshooting Scenario

Interviewer:

"A VM can communicate with another VM, but it cannot access the internet. What would you check?"

Reason systematically.
First:
Does the VM have a route toward external destinations?
Then:
Does it have a public IP, or is it supposed to use NAT?
If using Cloud NAT:
Is NAT configured correctly for the relevant subnet/region?
Then:
Are firewall rules allowing the outbound traffic?
Finally:
Is the destination itself reachable?
So the troubleshooting chain is:
VM
↓
Route
↓
NAT / Public IP
↓
Firewall
↓
Internet

---

26. Another Troubleshooting Scenario
Interviewer:
"VM A can communicate with VM B, but VM A cannot communicate with the database."
Think:
VM A
|
v
Routing
|
v
Firewall
|
v
Database
Check:

- Is the database reachable on its private IP?
- Is the correct port open?
- Are firewall rules permitting the source?
- Is the database actually listening?
- Is the application using the correct IP/DNS name?
- Is there a subnet/routing issue?

Again, don't randomly restart services.
Follow the network path.

---

27. The HPE Interview Chain
If the interviewer starts with:
"What is a VPC?"
you should be prepared for:
VPC → subnet → CIDR → route → private/public IP → firewall → ingress/egress → NAT → DNS → load balancer → health checks → multi-zone architecture → VPC peering → VPN → troubleshooting.
That is the complete mental chain.

Rapid Revision

What is a VPC?

A logically isolated virtual network for cloud resources.

What is a subnet?

An IP address range within a VPC/network.

What is a route?

A rule that determines where traffic destined for a particular network should go.

What is 0.0.0.0/0?

The IPv4 default route/address range representing all IPv4 destinations.

Public IP vs private IP?

Public IP provides external reachability; private IP is primarily for internal networking.

What is a firewall rule?

A rule controlling whether network traffic is permitted based on characteristics such as source, destination, protocol and port.

Ingress vs egress?

Ingress is incoming traffic; egress is outgoing traffic.

What is Cloud NAT?

A mechanism allowing resources without public IPs to initiate outbound internet connections.

Does NAT automatically allow inbound connections?

No.

Why use a load balancer?

To distribute traffic, improve availability, and provide a single entry point to backend services.

How does a load balancer detect unhealthy instances?

Through health checks.

VPC vs VPN?

VPC is the virtual network; VPN is a mechanism for securely connecting networks.

VPC peering?

Private connectivity between separate VPC networks.

The Core Mental Model

For the interview, remember this path:

Client
↓
DNS
↓
Public IP / Load Balancer
↓
Firewall
↓
VPC
↓
Subnet
↓
Private IP
↓
VM / Application
↓
Database

And for outbound traffic:

Private VM
↓
Route

↓
Cloud NAT
↓
Internet
If you can **trace a packet through this architecture and explain where it could fail**, you will be in very good shape for cloud-networking questions.

Part 4 — IAM, Authentication & Authorization
HPE SDE Interview Notes
IAM is one of the most important cloud-security topics to know because it sits at the intersection of **cloud infrastructure, application security, and access control**.
The key idea is simple:
IAM determines who or what can perform which actions on which resources.

---

1. What is IAM?
Question: What is IAM?
Answer:
IAM stands for **Identity and Access Management**. It is the system used to control access to cloud resources by determining **who or what is allowed to perform specific actions on specific resources**.
Think of IAM as:
WHO
+
WHAT CAN THEY DO
+
ON WHICH RESOURCE
For example:
Developer
↓
Can read
↓
Cloud Storage bucket
while:
Developer
↓
Cannot delete
↓
Production database
IAM exists to enforce these access-control decisions.

---

2. Authentication vs Authorization
This is an extremely common interview question.
Question: What is the difference between authentication and authorization?
Authentication determines:
Who are you?
Authorization determines:
What are you allowed to do?
For example, when you log into a system:
Username + Password
↓
Authentication
↓
"You're Manthan."

↓
Authorization
↓
"You can read this resource."
A useful memory trick:
Authentication = identity.
Authorization = permissions.

---

3. Identity in Google Cloud
An identity can represent a human or an application/workload.
For example:
Human
↓
User identity

Application
↓
Service account
This distinction is important.
A developer logging into Google Cloud and an application accessing Cloud Storage are **not the same type of identity**.

---

4. Users
A human user can authenticate to Google Cloud using an identity associated with their organization or Google account.
For example:
Developer
↓
Google Cloud
↓
IAM
IAM then determines what that user can access.
You shouldn't give every developer unrestricted access simply because they are developers.

---

5. Service Accounts
Question: What is a service account?
A **service account is an identity intended for applications, workloads, or services rather than human users.**
For example:
Backend Application
↓
Service Account
↓
Cloud Storage
The application can authenticate as that service account and receive the permissions assigned to it.
This is one of the most important concepts to understand.

---

6. Why Do We Need Service Accounts?
Suppose a backend application needs to read files from Cloud Storage.
A terrible design would be:
Backend
↓
Developer's credentials
↓
Cloud Storage

Why?
Because the application's access is now tied to a human identity.
If that developer leaves the organization, changes permissions, or rotates credentials, the application can break.
Instead:
Backend
↓
Dedicated Service Account
↓
Read-only access
↓
Cloud Storage
The application's identity is independent of the developer.

---

7. Least Privilege
Question: What is the principle of least privilege?
The principle of least privilege says:
An identity should receive only the permissions required to perform its intended task, and nothing more.
Suppose your application only needs to read objects from a bucket.
Bad:
Application
↓
Project Owner
This gives the application far more authority than necessary.
Better:
Application
↓
Service Account
↓
Read-only storage permission
This limits the potential damage if the application is compromised.

---

8. Roles
IAM permissions are commonly assigned through **roles**.
A role is essentially a collection of permissions.
Conceptually:
Role
|
+-- Permission A
+-- Permission B
+-- Permission C
Instead of assigning dozens of permissions individually, you assign an appropriate role.

---

9. Permission vs Role
This distinction is frequently tested.
Permission
A permission represents a specific allowed operation.
For example, conceptually:
storage.objects.get
means the identity can retrieve an object.
Role
A role is a collection of related permissions.
Role
|

+-- Permission 1
+-- Permission 2
+-- Permission 3
So:
Permissions are individual capabilities; roles bundle permissions together.

---

10. Types of IAM Roles
For interview purposes, understand three broad categories.
Basic Roles
Very broad roles such as:

- Owner
- Editor
- Viewer

They provide large sets of permissions.
Because they are broad, they are generally not ideal for fine-grained production access control.

Predefined Roles
Google provides roles designed for specific services and tasks.
For example, a role might provide access appropriate for working with Cloud Storage without granting unrelated administrative permissions.
These are usually preferable to broad basic roles.

Custom Roles
Organizations can create their own roles containing a specific set of permissions.
This provides greater control when predefined roles don't exactly match the required access model.

---

11. IAM Policy
Question: How does Google Cloud actually decide whether an identity can access something?
IAM policies bind **identities to roles on resources**.
Conceptually:
Identity
↓
Role
↓
Resource
For example:
Backend Service Account
↓
Storage Object Viewer
↓
Production Bucket
The result is:
The backend can perform the operations allowed by that role on that bucket.

---

12. IAM Hierarchy
Google Cloud resources have a hierarchy.
A simplified view is:
Organization
|
Folder
|
Project
|

Resource
For example:
Organization
|
+-- Folder: Engineering
|
+-- Project: Production
|
+-- VM
+-- Storage Bucket
+-- Database
IAM policies can be applied at different levels.

---

13. Why Does the Hierarchy Matter?
Suppose you give someone a role at the **project level**.
That permission can potentially apply to multiple resources within that project.
If instead you grant access at a specific resource level, the permission can be much narrower.
Therefore:
The scope at which you grant permissions matters.
This is directly related to least privilege.

---

14. Common Interview Scenario
Interviewer:
"Your backend needs to read files from one Cloud Storage bucket. How would you configure access?"
A strong answer:
"I'd create or use a dedicated service account for the backend and grant it the minimum predefined role necessary to read objects from that bucket. I would avoid giving it project-wide administrative permissions."
That answer demonstrates:

- Service accounts
- IAM
- Roles
- Least privilege
- Resource-level access

---

15. Another Scenario
Interviewer:
"Your application needs to read from Storage and write logs, but it doesn't need to modify Compute Engine instances. What permissions would you give it?"
Don't give:
Owner
Instead:
Application Service Account
|
+-- Storage read permission
|
+-- Logging permission
No Compute Engine administrative permissions.
This is exactly what least privilege means in practice.

---

16. Human vs Workload Identity
Question:
"Should an application use the developer's Google Cloud credentials?"
No.

Applications should generally use a workload-specific identity, such as a service account, rather than personal user credentials.
Why?
Because application identity should be:

- Independent of individual employees
- Auditable
- Revocable
- Permission-limited
- Appropriate for the workload

---

17. Authentication Scenario
Suppose your application wants to access Cloud Storage.
The process conceptually looks like:
Application
↓
Authenticate as service account
↓
Google Cloud IAM
↓
Check permissions
↓
Allow / Deny
The important distinction is:
Authentication establishes identity.
IAM authorization determines whether that identity has permission to perform the requested operation.

---

18. What Happens If IAM Denies Access?
Suppose:
Backend
↓
Cloud Storage
and the backend receives:
403 Forbidden
You should think:
Authorization problem.
Possible causes include:

- Incorrect service account
- Missing role
- Insufficient permissions
- Incorrect resource scope
- Incorrect IAM policy

This is different from a networking problem.

---

19. IAM vs Firewall
This is a **very important chained question**.
Question:
"What's the difference between IAM and a firewall?"
They control different things.
Firewall
Controls **network traffic**.
For example:
IP A
↓

TCP 443
↓
VM
The firewall determines whether that network traffic is allowed.
IAM
Controls **identity-based access to cloud resources and APIs**.
For example:
Service Account
↓
Cloud Storage API
IAM determines whether that identity is authorized to perform the requested operation.
So:
Firewall answers "Can this network traffic reach the resource?"
IAM answers "Is this identity allowed to perform this action?"
Both may need to permit an operation.

---

20. Example: Both Network and IAM Matter
Suppose your application accesses a database.
First:
Application
↓
Network
↓
Database
A firewall or network policy could block the connection.
Even if networking works:
Application
↓
Database
↓
Authentication / Authorization
the database itself may reject the identity.
Therefore:
Network access
AND
Application authorization
AND
Database authorization
may all be required.

---

21. IAM and Cybersecurity
This is an area where your cybersecurity background can help you answer more strongly.
A compromised application with excessive permissions creates a large blast radius.
Suppose:
Compromised Web App
↓
Owner Permissions
↓
Entire Cloud Project
An attacker may potentially manipulate many resources.
Compare:
Compromised Web App
↓

Restricted Service Account
↓
Read-only access to one bucket
The second architecture significantly reduces the potential impact.
That's why IAM is a major part of cloud security.

---

22. What is Role-Based Access Control?
Question: What is RBAC?
Role-Based Access Control assigns permissions through roles rather than individually assigning permissions to every user.
For example:
Developer
↓
Developer Role
↓
Permissions
Instead of:
Developer
↓
Permission 1
Permission 2
Permission 3
Permission 4
...
This makes access management easier to maintain.
Kubernetes also uses RBAC, so understanding this concept now will help later when we cover GKE.

---

23. IAM and Kubernetes
This is a likely chained question because your resume/interview track includes Docker and Kubernetes.
You may be asked:
"Is Google Cloud IAM the same as Kubernetes RBAC?"
No.
They operate at different layers.
Google Cloud IAM controls access to **Google Cloud resources and APIs**.
Kubernetes RBAC controls access to **Kubernetes resources within a cluster**.
For example:
Google Cloud IAM
↓
GCP resources
VMs, Storage, GKE APIs, etc.

Kubernetes RBAC
↓
Kubernetes resources
Pods, Deployments, Services, etc.
A real GKE environment can involve both.

---

24. Security Best Practices
For an interview, remember these principles:
Use dedicated service accounts for workloads.
Don't use personal credentials inside applications.
Apply least privilege.
Don't give Owner or Editor permissions simply because they're convenient.

Prefer specific predefined roles when they fit.
Use custom roles when necessary.
Limit the scope of permissions.
Resource-level access is preferable when project-wide access isn't required.
Audit access.
You should know which identity accessed which resource and when.

---

25. Interview Scenario: Compromised Service Account
Interviewer:
"Suppose an attacker compromises a service account used by your application. How do you reduce the damage?"
A strong answer:
"First, the service account should already have least-privilege permissions, so the attacker's capabilities are limited. I'd investigate the service account's activity, revoke or rotate credentials if applicable, remove unnecessary permissions, inspect audit logs, and determine whether other resources were accessed."
Notice the important security principle:
Least privilege doesn't prevent every compromise; it limits the blast radius of a compromise.
That's an excellent phrase to remember.

---

26. Rapid Revision
What is IAM?
A system for controlling which identities can perform which actions on which resources.
Authentication?
Determines who an identity is.
Authorization?
Determines what that identity is allowed to do.
What is a service account?
An identity intended for applications/workloads rather than human users.
What is least privilege?
Giving an identity only the permissions required for its task.
What is a permission?
A specific capability or operation an identity can perform.
What is a role?
A collection of permissions.
What is RBAC?
Access control based on assigning permissions through roles.
IAM vs firewall?
IAM controls identity-based authorization; firewalls control network traffic.
Should applications use developer credentials?
No. Use workload-specific identities such as service accounts.
What happens if IAM denies access?
The identity is not authorized for the requested operation, commonly resulting in an authorization error such as HTTP 403 when using an API.

Part 5 — Load Balancing, Autoscaling & High Availability

---

1. What is Load Balancing?
Question: What is a load balancer?
Answer:
A load balancer distributes incoming network traffic across multiple backend servers or instances.
Instead of clients connecting directly to one server:
Client → Server
we use:
+-- Server 1

```
          |
Client → Load Balancer → Server 2
          |
          +-- Server 3
```

The load balancer acts as the single entry point while distributing requests across available backends.

---

2. Why Do We Need a Load Balancer?

There are three major reasons.

Scalability

If one server cannot handle the traffic, add more servers and distribute requests between them.

Availability

If one server fails, traffic can be directed toward healthy servers.

Abstraction

Clients don't need to know the addresses of individual backend servers.

The client only communicates with the load balancer.

---

3. What Happens Without a Load Balancer?

Suppose:

```
Client
  |
  v
VM 1
```

VM 1 becomes overloaded:

CPU = 100%

or crashes completely.

The application becomes unavailable.

With a load balancer:

```
        +-- VM 1 ❌
        |
Client → LB -+-- VM 2 ✅
        |
        +-- VM 3 ✅
```

The healthy instances can continue serving requests.

---

4. How Does a Load Balancer Know Which Server Is Healthy?

Question: How does the load balancer know where to send traffic?

Through health checks.

A health check periodically tests whether a backend is functioning.

For an HTTP service, conceptually:

GET /health

If the backend responds correctly:

Healthy

If it repeatedly fails:

Unhealthy

The load-balancing system can then stop sending traffic to that unhealthy backend.

---

5. Health Check vs Application Monitoring

These aren't exactly the same thing.

A health check usually answers:

"Can this instance/service currently serve traffic?"

Monitoring is broader and can track:

- CPU

- **Memory**
- **Request latency**
- **Error rate**
- **Throughput**
- **Disk usage**
- **Application metrics**

A server could technically pass a basic health check while still having serious performance problems.
For example:
/health → 200 OK
but:
99th percentile latency = 8 seconds
The service is technically alive but clearly unhealthy from a user-experience perspective.

---

6. What is Autoscaling?
Question: What is autoscaling?
Autoscaling automatically changes the number of running instances based on workload or configured scaling signals.
For example:
Low traffic
↓
3 instances

Traffic increases
↓
6 instances

Traffic increases further
↓
10 instances
When demand falls:
10 instances
↓
5 instances
↓
3 instances
This is horizontal elasticity.

---

7. Autoscaling vs Vertical Scaling
Vertical scaling
Make an individual machine larger:
4 CPU / 8 GB
↓
16 CPU / 32 GB
Horizontal scaling
Add more machines:
1 VM
↓
3 VMs
↓
10 VMs
For highly available web applications, horizontal scaling is generally more useful because multiple instances also provide redundancy.

---

8. What Does Autoscaling Actually Solve?

Suppose your application normally receives:

100 requests/sec

and three instances can handle that comfortably.

Suddenly:

10,000 requests/sec

Three instances may not be sufficient.

Autoscaling can increase the number of instances.

But remember the important distributed-systems principle:

Scaling the application tier doesn't guarantee that the entire system can scale.

---

9. The Database Bottleneck Trap

Interviewer:

"Your web application automatically scales from 3 instances to 30. Why might the application still fail?"

Because another component may become the bottleneck.

For example:

30 Application Servers

|

v

Database

|

X

Connection limit

Suppose each application instance maintains 100 database connections.

Then:

30 × 100 = 3000 connections

If the database can only handle 500 connections, scaling the application makes the situation worse.

Therefore, when scaling a distributed application, consider:

- **Database capacity**
- **Connection pools**
- **Cache capacity**
- **External APIs**
- **Message queues**
- **Network bandwidth**
- **Storage throughput**

---

10. What is Autohealing?

Autoscaling and autohealing are different.

Autoscaling

Responds to workload.

Traffic ↑

↓

Instances ↑

Autohealing

Responds to instance failure.

VM 2 becomes unhealthy

↓

Health check detects failure

↓

VM 2 replaced/recreated

Remember:

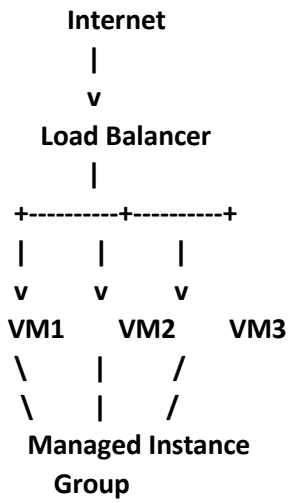
Autoscaling manages capacity. Autohealing manages failed instances.

---

11. Managed Instance Group + Load Balancer + Autoscaling

Now combine the concepts.

A production-style architecture might look like:



The components have different responsibilities:

Load Balancer: Distributes traffic.

Health Check: Determines whether backends are healthy.

Managed Instance Group: Maintains the VM fleet.

Autoscaling: Adjusts the number of instances.

Autohealing: Replaces unhealthy instances.

---

12. Complete Failure Scenario

Interviewer:

"You have three VMs behind a load balancer. VM2 crashes. What happens?"

A strong answer:

"The health check should detect that VM2 is unhealthy. The load balancer should stop routing traffic to that instance and continue sending requests to healthy instances. If the VMs are managed by a Managed Instance Group with autohealing enabled, the unhealthy instance can be recreated to restore the desired capacity."

Notice how many concepts you connected:

Health Check → Load Balancer → Managed Instance Group → Autohealing → High Availability

---

13. Another Failure Scenario

Interviewer:

"Traffic suddenly increases by 10x. What happens?"

Answer:

"If the application is configured for horizontal autoscaling, the managed instance group can increase the number of VM instances according to the scaling policy. The load balancer distributes traffic across the healthy instances."

Then expect:

"What if the database can't handle the additional traffic?"

Answer:

"Then the database becomes the bottleneck. I'd investigate CPU, memory, query latency, connection count and throughput. Depending on the workload, I could optimize queries, introduce caching, increase database capacity, use read replicas, or modify the architecture."

---

14. High Availability Architecture

Question: How would you make a Compute Engine application highly available?

Don't simply say:

"Use multiple VMs."

A stronger answer is:

"I'd deploy multiple application instances across independent zones, place them behind a load balancer, use health checks to remove unhealthy instances from traffic, and use a Managed Instance Group with autoscaling and

autohealing. I'd also ensure that stateful dependencies such as databases and storage have an appropriate high-availability and recovery strategy."
The last sentence is important.
Your application tier being highly available doesn't automatically make your database tier highly available.

---

15. Multi-Zone Architecture
Suppose everything is in one zone:

```
Region
 |
 +-- Zone A
     |
     +-- VM1
     +-- VM2
     +-- VM3
```

If Zone A fails:
VM1 ❌
VM2 ❌
VM3 ❌
The entire application may become unavailable.
Better:

```
Region
 |
 +-- Zone A → VM1
 |
 +-- Zone B → VM2
 |
 +-- Zone C → VM3
```

A failure affecting one zone doesn't necessarily take down all application instances.

---

16. Load Balancer vs Reverse Proxy
Question: Is a load balancer the same as a reverse proxy?
Not necessarily.
A reverse proxy is a server that receives client requests and forwards them to backend servers.
A load balancer distributes traffic among multiple backends.
There is overlap because many load balancers can perform reverse-proxy functionality.
The concepts shouldn't be treated as identical.

---

17. Layer 4 vs Layer 7 Load Balancing
This is a useful networking follow-up.
Layer 4
Operates primarily using transport-level information such as:

- **TCP**
- **UDP**
- **IP**
- **Port**

It doesn't need to understand the application-level HTTP request.
Layer 7
Understands application-level protocols such as HTTP/HTTPS.
It can make decisions using information such as:
/path
/ API
/ images

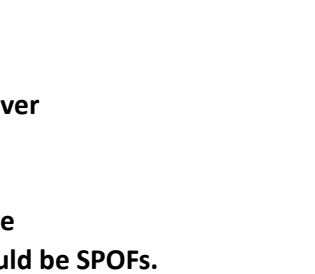
For example:
/api/* → Backend service
/static/* → Static content service
This is why Layer 7 load balancing can perform more application-aware routing.

---

18. What if the Load Balancer Fails?
This is another classic interview chain.
Interviewer:
"You've made your servers redundant. What happens if the load balancer itself fails?"
The answer is that production cloud load-balancing services are themselves designed as managed, distributed infrastructure rather than a single VM sitting in front of your application.
The broader architectural principle is:
Don't introduce a single point of failure while trying to remove another one.
This principle applies everywhere in distributed systems.

---

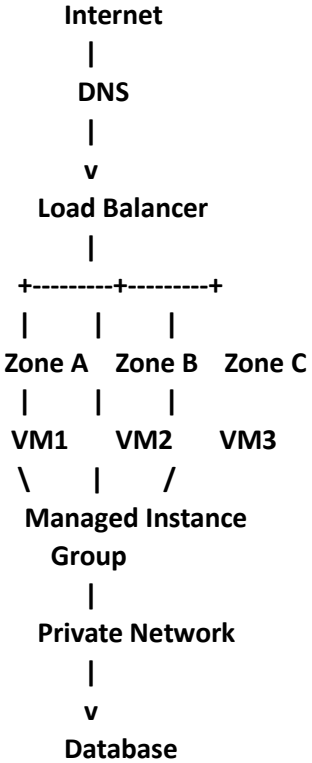
19. What is a Single Point of Failure?
A Single Point of Failure (SPOF) is a component whose failure can bring down the entire system.
Example:



Both could be SPOFs.
A more resilient architecture introduces redundancy where required.

---

20. End-to-End Architecture
Now combine everything from the entire GCP track:



Around this architecture:
VPC provides the network.
Subnets provide IP ranges.

Firewall rules control network traffic.
IAM controls access to cloud resources/APIs.
Health checks detect unhealthy backends.
Load balancing distributes traffic.
Autoscaling adjusts capacity.
Autohealing replaces failed instances.
Multi-zone deployment improves availability.
This is the architecture you should have in your head when answering scenario questions.

---

21. The Ultimate HPE Scenario
Interviewer:
"Design a highly available web application on Google Cloud."
Don't panic and start listing services.
Walk through the requirements.
Step 1 — Application
Containerized/stateless web application.
Step 2 — Compute
Use either Compute Engine + Managed Instance Groups or a managed platform such as Cloud Run depending on the requirements.
Step 3 — Traffic
Use a load balancer for the VM-based architecture.
Step 4 — Availability
Deploy instances across multiple zones.
Step 5 — Scaling
Use horizontal autoscaling.
Step 6 — Failure Recovery
Use health checks and autohealing.
Step 7 — Networking
Keep internal services on private networking and expose only the necessary public entry point.
Step 8 — Security
Use least-privilege IAM and dedicated service accounts.
Step 9 — Database
Use an appropriate managed database with its own HA/backup/recovery strategy.
Step 10 — Monitoring
Monitor:

- **CPU**
- **Memory**
- **Request rate**
- **Latency**
- **Error rate**
- **Instance health**
- **Database health**

This gives you a complete engineering answer rather than a collection of GCP buzzwords.

---

22. Final GCP Interview Revision
You now need to be comfortable with these chains:
Cloud Fundamentals
Cloud → IaaS/PaaS/SaaS → scalability → elasticity → availability → regions/zones → managed services
Compute
Compute Engine → VM → hypervisor → machine type → boot disk → persistent disk → public/private IP → SSH
Networking
VPC → subnet → CIDR → route → firewall → ingress/egress → NAT → DNS → load balancing
Security

IAM → authentication → authorization → service accounts → permissions → roles → least privilege → RBAC
Availability
Load Balancer → health check → Managed Instance Group → autoscaling → autohealing → multi-zone architecture

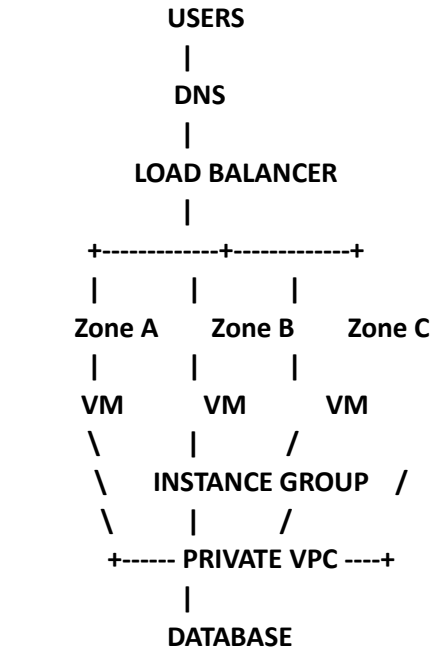
---

The 15 Questions I'd Expect You to Survive

1. **What is cloud computing?**
2. **IaaS vs PaaS vs SaaS?**
3. **Compute Engine vs Cloud Run vs GKE?**
4. **What is a VPC?**
5. **VPC vs subnet?**
6. **Public IP vs private IP?**
7. **What is a firewall rule?**
8. **What is IAM?**
9. **Authentication vs authorization?**
10. **What is a service account?**
11. **What is least privilege?**
12. **What is a load balancer?**
13. **Autoscaling vs autohealing?**
14. **How would you make an application highly available?**
15. **A production application is down — how would you troubleshoot it?**

If you can answer those without memorized definitions and can survive 2–3 follow-ups on each, your GCP knowledge is sufficient for the level of questioning I'd expect from an HPE SDE interview.

Final Mental Model
Keep this in your head before the interview:



Then attach the concepts:
VPC → connectivity
Firewall → network access control
IAM → identity/access control
LB → traffic distribution
Health Check → detect failure
MIG → manage VM fleet
Autoscaling → handle changing demand
Autohealing → recover failed instances
Zones → fault isolation

Monitoring → observe system behavior

That's the GCP portion I'd stop at for now. Don't burn your remaining preparation time memorizing obscure GCP services. For an HPE SDE interview, your higher ROI now is to connect this knowledge to Docker/Kubernetes, Linux, networking, OS, DBMS and system-design scenarios—because that's where the interviewer can turn one cloud question into a 10-minute technical chain.
